ECOBRIO

Complete System Architecture, Workflow & Implementation Blueprint

Distributed Reverse-Logistics • Geospatial Telemetry • Green Incentive Clearing

A consolidated engineering design document derived from the EcoBrio PRD and the subsequent architecture reviews.

Primary language: Go • Data: PostgreSQL + Redis • Local event services: LocalStack

Purpose: buildable architecture, testable invariants, explicit failure paths, reproducible evidence

Prepared 10 October 2026 | Source PRD reproduced in Appendix A

Document Control and How to Use This Blueprint

Item	Details
Document purpose	Consolidate the original EcoBrio PRD and the architecture/design reviews into one implementation-ready engineering reference.
Source baseline	EcoBrio: System Architecture & PRD, 4 pages. The original PDF is appended verbatim in Appendix A.
Status	Architecture proposal and implementation plan. Proposed safeguards and recommendations are explicitly identified as such.
Primary audience	Project builder, code reviewers, evaluators, and technical interviewers.
Key constraint	Designed for a zero-budget local development setup using Docker and LocalStack; optional external AI and map services still depend on their provider/network.

Source discipline

PRD requirements and targets are reproduced as stated. Architectural safeguards added during review are recommendations. Performance targets are not presented as achieved measurements unless a real benchmark report proves them.

Contents

1. Executive Summary and Project Definition
2. Scope and Requirement Traceability
3. Technology Stack and Engineering Techniques
4. Architecture Principles and System Ownership
5. Master System Architecture
6. End-to-End User Workflow
7. Booking Protocol and Failure Recovery
8. Credit Clearinghouse and Ledger
9. Event-Driven Processing
10. WAL Durability and Recovery
11. Geospatial Incidents and WebSockets
12. KachraSeth AI Triage
13. Data Model, API, and Event Contracts
14. Security, Privacy, and Reliability
15. Observability and Performance Benchmarks
16. Test Strategy and Acceptance Matrix
17. Ten-Day Implementation Roadmap
18. Risk Register and Architecture Decisions
19. Repository Structure and Local Developer Workflow
20. Demo Plan, Interview Positioning, and Delivery Checklist
21. References and Glossary
Appendix A. Original EcoBrio PRD

1. Executive Summary and Project Definition

EcoBrio is a distributed reverse-logistics platform for coordinating municipal e-waste collection hubs, booking limited collection capacity, verifying recyclable material, settling green incentives, ingesting smart-bin telemetry, and tracking waste incidents on a real-time map. It is also intentionally an engineering portfolio project: its value comes from correctness under concurrency, durable ingestion, explicit recovery behavior, observability, and repeatable tests.

The original PRD positions the project toward engineering audiences associated with Rubrik, JusPay, Myntra, and Booking.com. Its distinctive technical themes are Go concurrency, atomic Redis scripts, PostgreSQL transactions and optimistic locking, a custom segmented write-ahead log (WAL), geospatial indexing, event-driven processing, and structured AI tool calling.

> **Core thesis**
>
> Build one coherent application that can demonstrate three claims with evidence: (1) bookings cannot exceed capacity, (2) acknowledged durable sensor events can be recovered and replayed safely, and (3) credit/incident operations remain consistent across retries and asynchronous failures.

1.1 Problem statement

Urban circular economies can suffer from irregular recycler routing, hazardous e-waste contamination, and inaccurate incentive allocation. EcoBrio intends to give citizens and municipal operators a common system for booking collection, verifying submitted material, issuing auditable incentives, tracking incidents, and ingesting telemetry from smart equipment.

1.2 Product goals

- Prevent reservation overselling when many users compete for a small remaining capacity.
- Prevent duplicate green-credit effects when QR scans and network requests are retried.
- Provide an auditable credit ledger and safe compensation for failed downstream operations.
- Build a custom WAL capable of validating, replaying, and recovering sensor events after process termination.
- Provide real-time geospatial incident visibility with reliable reconnect behavior.
- Use an AI triage agent only as a validated interface to deterministic backend tools.
- Make performance and reliability claims reproducible using automated test scripts and monitoring.

1.3 Product boundary

In scope for the core prototype	Explicitly not required for the initial local build
Go backend; citizen/collector/admin web views; booking; submission inspection; credit ledger; incident lifecycle; sensor ingestion; WAL; Redis GEO; WebSockets; SQS/S3 emulation; monitoring and benchmarks.	Production multi-region deployment; real municipal integrations; real monetary payments; blockchain; native mobile apps; Kubernetes; live production AWS services unless a later deployment requirement justifies them.

2. Scope and Requirement Traceability

This section separates what the source PRD directly specifies from additional engineering recommendations introduced during review. The original targets are goals, not proof of achieved behavior.

2.1 Source requirements and targets

Area	PRD baseline	Implementation interpretation / addition	Classification
Core backend	Go (Golang 1.22+)	Pin a compatible toolchain and dependencies.	Source requirement
Relational data	PostgreSQL 16; ACID ledger; optimistic locking	Use explicit version columns, database constraints, and transactional business invariants.	Source + clarification
Fast coordination / geo	Redis 7; Lua capacity scripts; GEO; Pub/Sub	Define which state is authoritative vs. derived/cached.	Source + clarification
Cloud emulation	LocalStack S3 and SQS	Initialization scripts, queue config, consumer retry policy.	Source + recommendation
Durable telemetry	Custom segmented WAL; length + CRC32 + payload; batch fsync; 10 MB rotation; snapshots; replay	Versioned records, event IDs, safe checkpoints and explicit durability acknowledgement.	Source + recommendation
Booking correctness	Two-layer Redis/Postgres approach; up to 3 SQL retries; HTTP 409; no over-allocation in 2,000 attempts	Reservation identity; safe compensation; reconcile uncertain commits; SQL must independently enforce capacity.	Source + critical refinement
Credits	Idempotency key; FSM; escrow; compensation if voucher issuance fails	Persist idempotency in PostgreSQL; define voucher vs. earning semantics; make consumers idempotent.	Source + critical refinement
Incident updates	Redis GEO; WebSocket hub; Leaflet / OpenStreetMap; 10,000+ connections; <15 ms broadcast target; GEO <2 ms target	Define query size, broadcast fan-out, latency boundary, reconnect behavior, and benchmarks.	Source + clarification
AI triage	Gemini Free / Groq; tools report_waste_incident, find_nearest_hub, get_user_credits; asynchronous queue dispatch	Tool allowlist, schema validation, authorization, timeout, deterministic form fallback.	Source + recommendation
Observability	Prometheus + Grafana; p99 latency, throughput, WebSockets, lock collisions	Map each target to a named metric and raw test artifact.	Source + clarification
Performance targets	>= 3,000 requests/sec; p99 <40 ms; 0% over-allocation; 100% durable-record integrity; <15 ms live broadcast; zero races	Specify workload, environment, duration, error mix and measurement window.	Source + clarification

2.2 Success targets from the PRD

Metric	PRD target	Verification named by PRD	Status in this document
Peak booking throughput	At least 3,000 requests/sec	k6; 500 virtual users against localhost	Target only; not claimed achieved
Reservation API p99	Under 40 ms	Prometheus histogram ecobrio_request_duration_seconds	Target only; benchmark specification required
Concurrency safety	0% over-allocation	k6 race test against last five slots	Required invariant
Durability and recovery	100% byte integrity / zero loss	Terminate with kill -9 during 5,000 sensor writes; restart and recover	Requires acknowledgement boundary definition
Live broadcast latency	Under 15 ms end-to-end	Benchmarked WebSocket client listener	Requires fan-out and latency-boundary definition

Metric	PRD target	Verification named by PRD	Status in this document
Code concurrency safety	Zero detected data races	go test -race ./... in CI/CD	Test gate, not a formal proof of all concurrency correctness
Redis GEO query	Sub-2 ms stated in detailed engine section	Not fully defined in success table	Add dataset size, search radius, results, percentile
Persistent WebSocket capacity	10,000+ stated in detailed engine section	Benchmark client telemetry mentioned	Add connection establishment, traffic and memory protocol
Redis-side rejection	Deflect 95% of invalid requests before SQL	Described in booking engine	Define “invalid” request and count source

Important claim-control rule

The source PRD’s interview pitch says the benchmarks were validated. The PRD itself also lists them as targets and future verification gates. Until raw benchmark reports exist, describe these numbers as targets; replace them with measured values only after testing.

3. Technology Stack and Engineering Techniques

3.1 Required / PRD-aligned stack

Layer	Technology	Use in EcoBrio	Status
Backend	Go 1.22+	HTTP API, domain logic, goroutines, channels, WebSocket hub, workers, WAL.	Required
Relational database	PostgreSQL 16	Authoritative bookings, slots, incidents, ledger, request outcomes, outbox and checkpoints.	Required
In-memory and geospatial	Redis 7 Alpine	Lua, capacity reservation, GEOADD/GEOSEARCH, Pub/Sub, fast cache.	Required
Local cloud emulation	LocalStack S3 / SQS	Local queueing, event buffering, optional artifact/object storage.	Required
Durable storage	Custom segmented Go WAL	Append-only records, CRC32, batch sync, rotation, snapshots, replay.	Required
Observability	Prometheus + Grafana	Metrics, histograms, throughput, active connections, lock collisions, recovery.	Required
Load testing	k6	Controlled arrival rate, booking race, mixed traffic, API latency.	Required
Frontend map	Leaflet.js + OpenStreetMap tiles	Incident map, color-coded lifecycle state, viewport interactions.	Required
Real-time transport	Go WebSockets	Live incident updates with backpressure and reconnect/resync.	Required / implied
AI provider	Gemini Free or Groq API	Structured tool calling for citizen reports.	Required option in PRD

Layer	Technology	Use in EcoBrio	Status
Version control / delivery	Git + GitHub; GitHub Actions recommended	Reviewable history, automated tests, reproducible release.	Recommended

3.2 Suggested Go packages and tools

Need	Suggested option	Notes
HTTP routing	net/http or chi	Prefer standard library first unless middleware routing complexity justifies chi.
PostgreSQL driver	pgx	Use transactions and typed query errors.
Redis client	go-redis	Keep scripts in version-controlled files and test their return contracts.
SQL migrations	golang-migrate or equivalent	Migrations should run deterministically from a clean database.
Structured logs	log/slog	Include request ID, event ID, transaction ID, result and latency; avoid PII.
Metrics	prometheus/client_golang	Use bounded-label histograms/counters/gauges.
WebSockets	A maintained Go library or carefully implemented protocol	Keep one writer per connection; isolate slow clients.
API contract	OpenAPI	Document request IDs, idempotency behavior, errors and state transitions.
Configuration	Environment variables + .env.example	Never commit actual credentials.
CI	GitHub Actions	Run formatting, unit tests, race tests and selected integration tests.

3.3 Engineering techniques to demonstrate

- Concurrency control: goroutines, channels, mutexes only where needed, bounded worker pools, context cancellation, backpressure.
- Transactional state changes: SQL transactions, constraints, conditional updates, row versioning and explicit failure mapping.
- Idempotency: stable request identities, request fingerprinting, unique transaction IDs and stored outcomes.
- Distributed workflow reliability: transactional outbox, at-least-once queue consumption, deduplication, bounded retries and reconciliation.
- Storage durability: binary record encoding, CRC32, batch fsync, torn-tail detection, safe checkpoints and replay.
- Observability: structured logs, metrics, latency histograms, request/event correlation and benchmark evidence.
- Testing: table-driven unit tests, integration tests against real local services, property/invariant tests, race detection, load testing and fault injection.

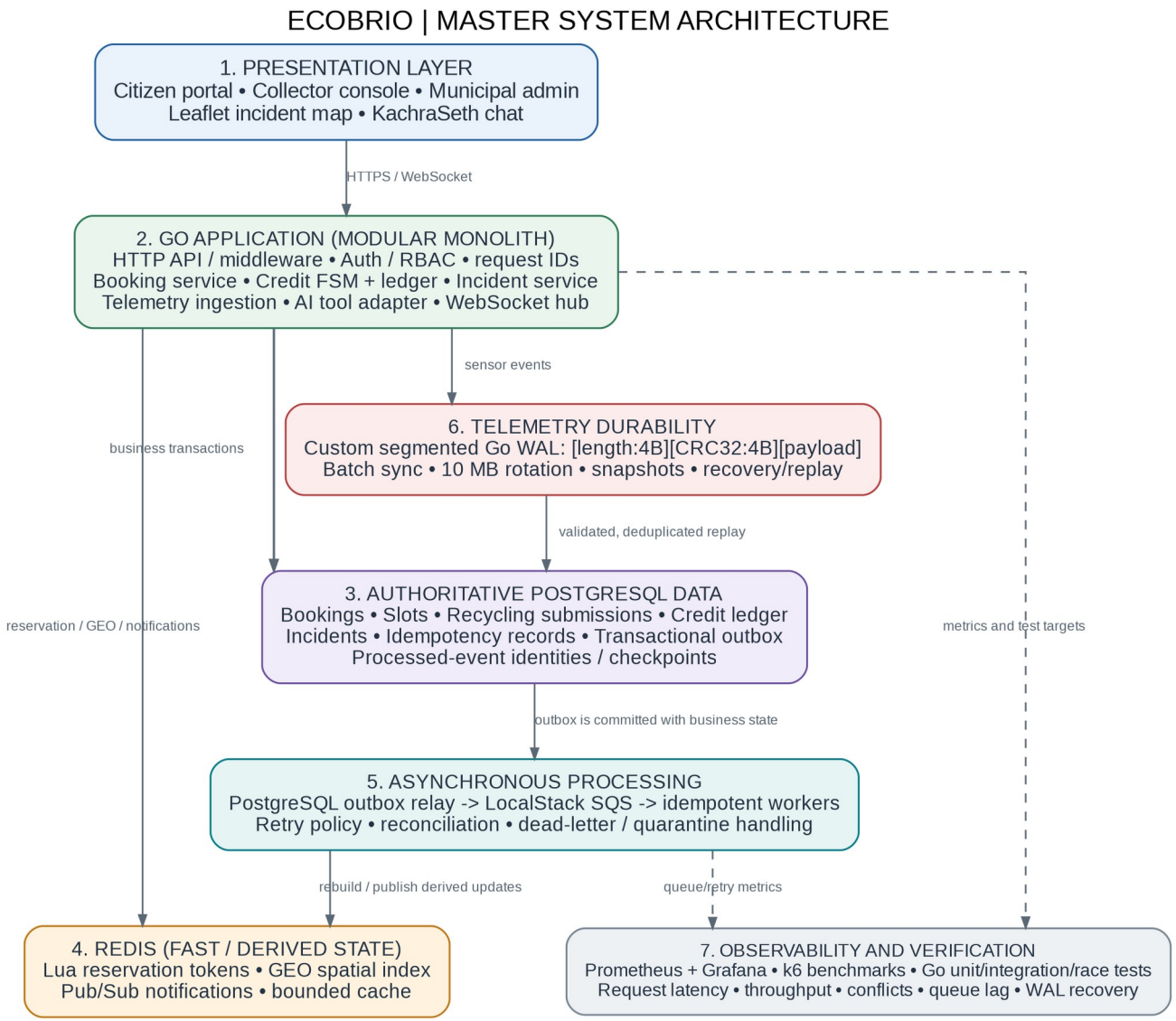
4. Architecture Principles and State Ownership

Principle	Implementation meaning
PostgreSQL owns confirmed business state.	Confirmed bookings, credit ledger postings, incident lifecycle, idempotency outcomes, outbox events and processed event IDs are durable facts.
Redis is fast, but not a cross-store transaction manager.	Lua can make a Redis script atomic; it cannot atomically commit PostgreSQL. Redis tokens and GEO indexes need reconciliation or rebuild paths.
Asynchronous messages may be duplicated.	Use stable event IDs, idempotent business effects and bounded retry/dead-letter policies.
WAL durability needs a stated acknowledgement contract.	Only acknowledge durable receipt after the configured sync boundary succeeds; distinguish durable from database-applied.
AI proposes; deterministic code disposes.	Model output is untrusted. Backend code validates schema, location, scope, authorization and workflow state before acting.
Real-time transport is not the source of truth.	WebSockets send updates; reconnecting clients recover current state from an authoritative snapshot or cursor-based resync.
Modular monolith first.	Separate domain packages and worker loops in one deployable Go backend reduce deployment overhead while preserving responsibility boundaries.
Targets are measured, not assumed.	Every performance and reliability statement must link to an executable test and raw output.

4.1 Recommended service boundaries

Module	Owns	Must not own
booking	Slot availability rules, reservation token lifecycle, booking idempotency and reconciliation.	Credit issuance; AI prompt logic.
credits / ledger	Submission FSM, escrow/redeem rules, balanced ledger posting, reversal and audit.	Redis as sole balance authority.
telemetry / WAL	Record format, append, sync, segment rotation, recovery and replay.	WebSocket business delivery semantics.
incidents / geospatial	Incident domain transitions, authorized assignment, coordinate validation, GEO projection.	Sole truth stored only in Redis.
queue / outbox	Event envelope, outbox relay, worker idempotency, retry and failure inspection.	Uncoordinated dual-write after business commit.
AI adapter	Provider calls, tool schema, timeout/fallback and proposed structured data.	Direct unvalidated privileged mutations.
observability	Metrics, logs, dashboards and benchmark artifact capture.	High-cardinality personal identifiers as metric labels.

Figure 1. Master System Architecture

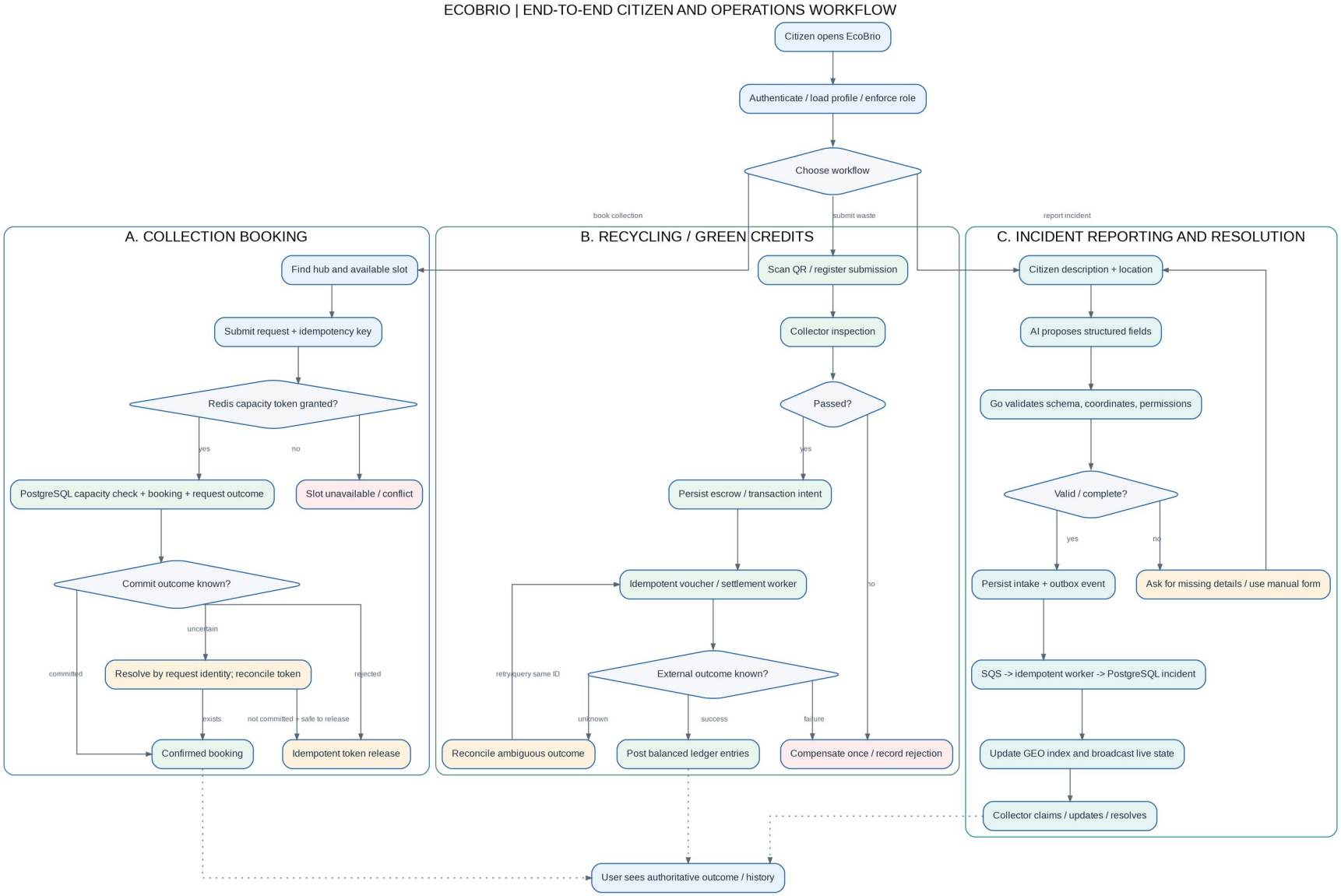


Logical architecture. PostgreSQL owns business truth; Redis supports fast/derived state; the outbox/SQS path is asynchronous; the custom WAL protects telemetry ingestion.

5. Component Responsibilities and Key Interfaces

Component	Inputs	Outputs / side effects	Critical guarantee
HTTP API / middleware	HTTP requests, identity/session, headers	Validated service calls, consistent errors, request IDs	Authorization and validation happen before state mutation.
Booking service	Slot ID, citizen ID, idempotency key	Booking state, reservation token outcome	Capacity never exceeded; uncertain commit resolved by request identity.
Credit clearinghouse	QR submission, inspection result, idempotency key	FSM transitions, escrow/redeem intent	One logical effect per business event; illegal transitions rejected.
Ledger	Account, transaction ID, integer points	Balanced immutable entries, balance projection	Atomic balanced posting; corrections via explicit reversal/adjustment.
Telemetry ingestion / WAL	Sensor ID, event ID, timestamp, payload	WAL record, durable acknowledgement status	Acknowledgement matches the documented durability boundary.
Incident service	Description, coordinates, severity, authorized actor	Persisted lifecycle change and event/outbox entry	Only valid authorized transitions commit.
Outbox relay	Pending outbox rows	SQS messages and publication status	Retryable publication; duplicate publication is tolerated.
Worker	SQS event with event ID	Database effect and processed-event identity	Deduplication + business effect commit together where possible.
WebSocket hub	Committed update event / subscription	Live message delivery	Slow clients isolated; snapshots/resync recover missed events.
AI adapter	Citizen narrative, permitted tool set	Proposed typed action / clarification request	LLM output never bypasses backend authorization and validation.

Figure 2. End-to-End Citizen and Operations Workflow



Three business journeys: booking; recycling submission and green credits; AI-assisted incident reporting and resolution. All paths end in a persisted, inspectable outcome.

6. End-to-End User Workflow

6.1 Collection booking journey

1. Citizen authenticates and locates a suitable collection hub.
2. The application displays available slots. Client submits a booking request with a stable idempotency key.
3. Redis executes a Lua script to attempt a capacity reservation and returns a unique reservation token or an unavailable result.
4. PostgreSQL independently checks the capacity invariant, creates the booking and stores the request outcome in a transaction.
5. If commit is confirmed, the Redis token is marked confirmed. If the SQL transaction is definitely rejected, the token is released idempotently.
6. If commit outcome is uncertain, the service resolves by the request key before releasing capacity; reconciliation repairs any leftover token/counter mismatch.
7. The citizen sees confirmed, unavailable, or pending/retry-safe outcome. A pending outcome is not falsely presented as success.

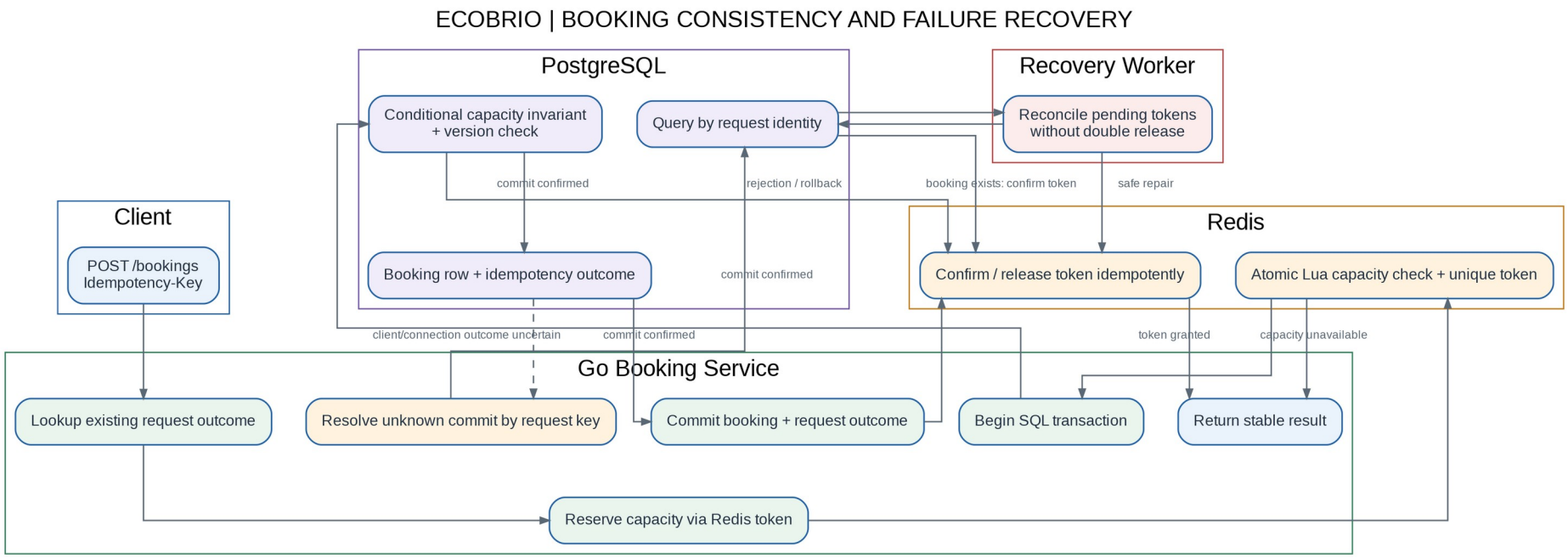
6.2 Recycling and credit journey

1. Citizen scans a QR passport or registers a recycling submission.
2. Backend validates the submission identity, duplicate state, and eligible account.
3. Collector performs material inspection and records pass/fail with a reason.
4. Passing submissions enter the defined credit/escrow workflow. The exact voucher semantics must be decided before implementation.
5. A durable event is published using the outbox, then processed by a worker that deduplicates by business transaction ID.
6. Ledger posting, final state and audit history are committed consistently. Ambiguous downstream outcomes enter reconciliation rather than immediate blind rollback.
7. Citizen sees the persisted transaction outcome and current history.

6.3 Incident journey

1. Citizen enters a natural-language report and a location or map pin.
2. KachraSeth proposes structured fields and asks for clarification when needed.
3. Go validates the payload, coordinates, user permissions and idempotency before accepting it.
4. The accepted intake and outbox event are committed. A relay publishes the event to LocalStack SQS.
5. Worker creates the incident once, commits it in PostgreSQL and updates the Redis GEO projection.
6. WebSocket hub broadcasts to relevant clients. A collector claims and progresses the incident using authorized state transitions.
7. Clients reconnecting later obtain a fresh snapshot or resume from a documented event cursor.

Figure 3. Booking Consistency and Failure Recovery



The Redis reservation token is not itself a committed booking. A database invariant, stored idempotency outcome, safe release and reconciliation are required.

7. Booking Protocol and Failure Recovery

7.1 The cross-store consistency issue

A Redis Lua script is atomic inside Redis. A PostgreSQL transaction is atomic inside PostgreSQL. The two operations do not share a single atomic commit. Redis may decrement capacity and SQL may fail; SQL may commit but the response can be lost. A naive unconditional increment on every SQL error can restore capacity twice or release capacity for a booking that already committed.

7.2 Recommended booking algorithm

1. Validate request and caller; resolve an existing idempotency key and return its stored outcome if present.
2. Ask Redis Lua to reserve capacity using a unique reservation token. The Lua script must record enough token state to make duplicate claims and releases safe.
3. Open a PostgreSQL transaction. Enforce capacity in the database with a conditional update/transactional check, use the slot version where appropriate, insert the booking, and store the idempotency outcome atomically.
4. On confirmed commit, mark the Redis token confirmed. The confirmation step may be retried safely.
5. On confirmed rollback/rejection, release the token idempotently. A second release must not increment capacity twice.
6. On connection loss or uncertain commit, query by request identity. If booking exists, return the original result and reconcile the token as confirmed.
7. If the final outcome is not established, mark reconciliation work and permit safe client retry; do not claim success or blindly release the token.
8. Periodically reconcile active/pending Redis tokens and capacity counters against confirmed PostgreSQL bookings.

7.3 Required invariants

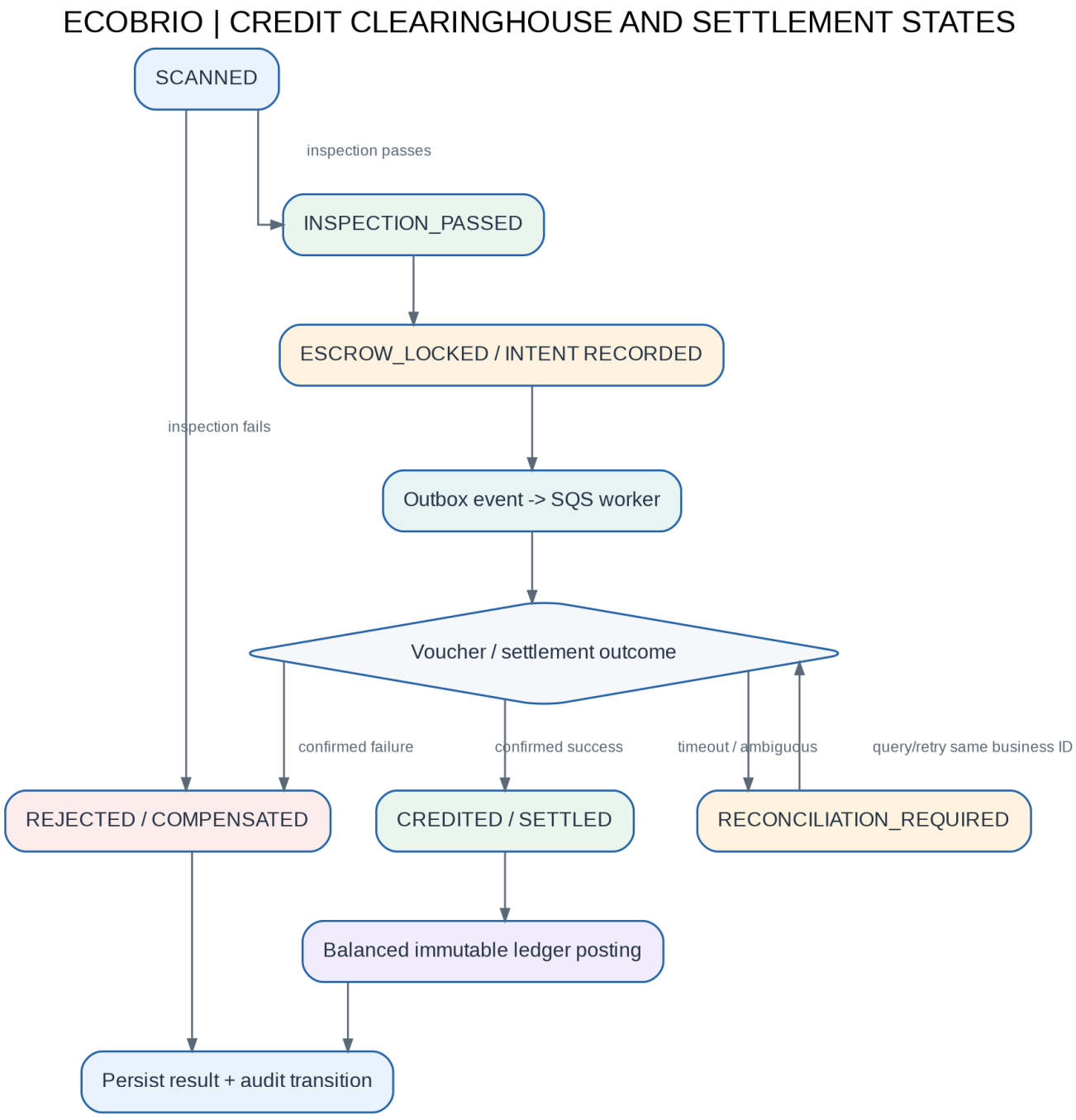
Invariant	Enforcement / test
Confirmed bookings never exceed slot capacity.	Database constraint/conditional transaction; race test with 2,000 attempts for five slots.
One idempotency key identifies one logical request outcome in its scope.	Unique key + request fingerprint; replay test and same-key/different-payload test.
A reservation token is claimed and released at most once logically.	Unique token identity; token lifecycle; duplicate release tests.
Redis may be stale but cannot overrule a confirmed PostgreSQL booking.	Reconciliation from durable booking state; Redis loss/drift test.
Uncertain commit is not treated as rollback.	Inject timeout immediately after commit; retry returns stored outcome.
Retries are bounded.	Maximum three optimistic conflict retries per PRD; return a clear HTTP 409 or retryable failure according to outcome.

7.4 Failure matrix

Failure injection	Danger if handled naively	Required behavior
Redis token acquired; SQL transaction rejected	Phantom full slot / lost capacity	Idempotent token release and periodic reconciliation.

Failure injection	Danger if handled naively	Required behavior
SQL commits; client response lost	Duplicate booking on retry	Resolve by persisted idempotency key and return existing result.
SQL commit result unknown	Blind release could oversell	Query by request identity; if unresolved, pending reconciliation.
Redis restarts or counter is lost	Incorrect availability responses	Rebuild/reconcile derived counters from durable bookings and active reservation state.
Two callers use same idempotency key	Duplicate side effects or ambiguous response	Persist key+fingerprint+outcome; reject conflicting payload reuse.
Reservation expires while SQL is still resolving	Token release may race a successful commit	Leases must account for transaction state; expiration triggers reconciliation, not blind capacity release.

Figure 4. Credit Clearinghouse and Settlement States



The principal PRD FSM is retained; operational failure and reconciliation states are explicit additions. Choose the voucher/credit interpretation before building.

8. Credit Clearinghouse and Ledger

8.1 PRD state machine

The original state transitions are SCANNED → INSPECTION_PASSED → ESCROW_LOCKED → CREDITED. Illegal transitions must fail with typed errors. Client calls use X-Idempotency-Key; repeated requests must not mint duplicate credits. Voucher issuance failure triggers compensating behavior in the original design.

8.2 Unresolved business decision: earning vs. redemption

Option	Meaning	Impact
A. Voucher is part of earning	Inspection, escrow and voucher issuance are one reward settlement. A confirmed voucher failure may compensate the escrow according to the defined policy.	Closest to the described PRD saga, but credit earning depends on voucher availability.
B. Earning and redemption are separate	Inspection earns points into a ledger. Later voucher redemption is a separate transaction that places/removes a hold and issues a voucher.	Often cleaner wallet semantics, but requires a separate redemption FSM and a PRD update.

Decision required before implementation

Pick Option A for strict fidelity to the described flow, or Option B if earned points should remain available independently of voucher delivery. Do not mix their compensation rules. In either option, resolve a timeout by querying/retrying the same external business identity before compensating.

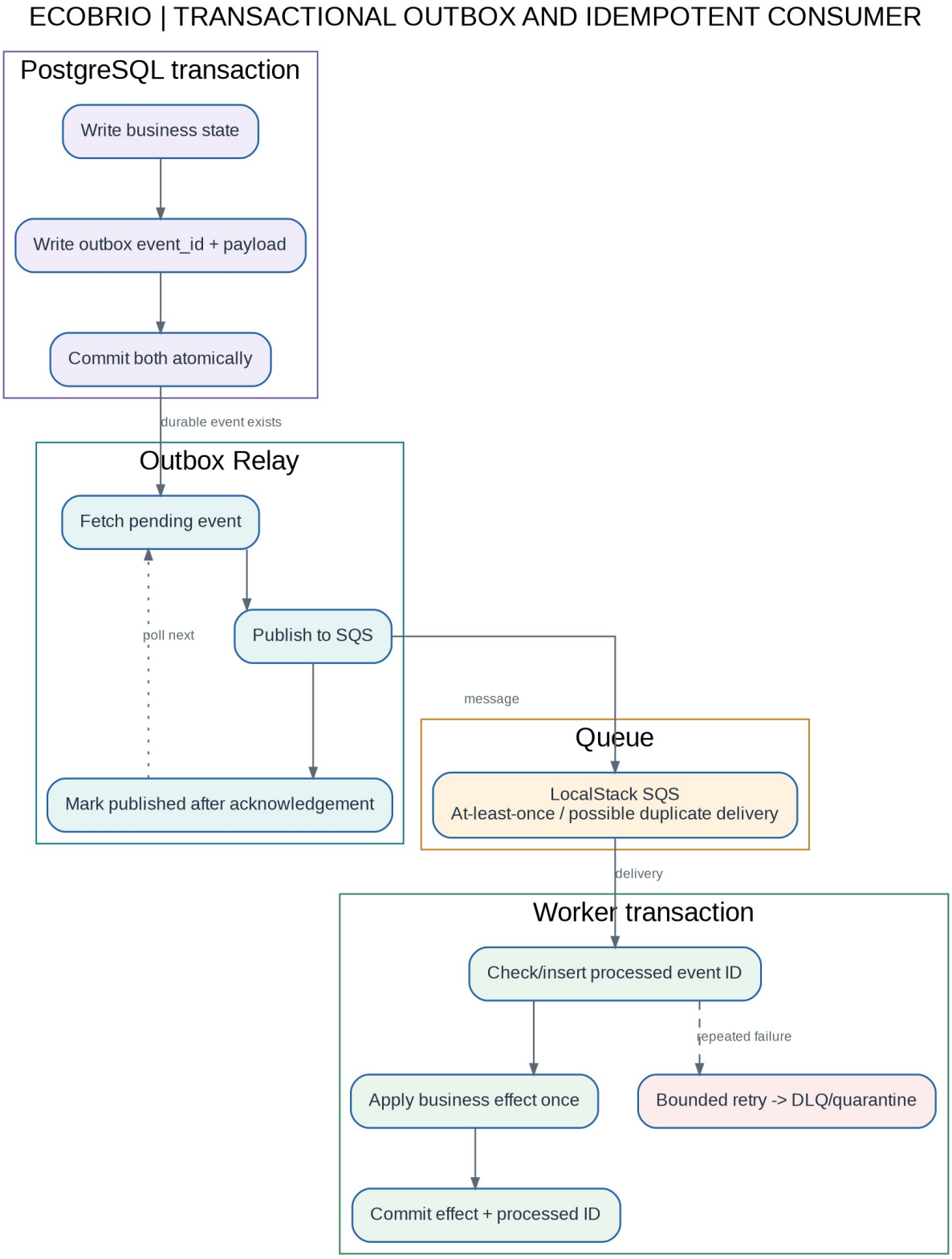
8.3 Ledger rules

- Use integer point units or another exact fixed-precision representation, not floating-point balances.
- Post all entries for one transaction in a single PostgreSQL transaction; enforce balanced debit/credit postings at the service and transaction boundary.
- Persist request idempotency in PostgreSQL so Redis cache loss cannot mint credit twice.
- A repeated key with the same request returns the stored result; the same key with a materially different request payload is rejected.
- Ledger postings are immutable after commit. Corrections are explicit reversal/adjustment transactions with references to the original posting.
- Persist each FSM transition and actor/action metadata for auditability.
- Voucher and queue side effects use stable transaction IDs and idempotent consumers.
- If voucher outcome is ambiguous, use a reconciliation-required state; do not blindly issue a second voucher or reverse points.

8.4 Example: ambiguous voucher result

Suppose a redemption reserves 20 points and the citizen has 100 available points. The voucher provider may have issued the voucher even if its response timed out. EcoBrio should query the provider or retry with the same external idempotency identity. Releasing the hold immediately could leave the citizen with both a valid voucher and the restored points. If failure is confirmed, release escrow/hold once; if success is confirmed, settle the ledger once.

Figure 5. Transactional Outbox and Idempotent Consumer



The business state and outbox intent commit together. Relay and worker delivery can retry; duplicate messages are expected and handled idempotently.

9. Event-Driven Processing

9.1 Event envelope

Use a versioned event envelope for all asynchronous messages. Keep the envelope stable across publishers and consumers; keep domain payloads explicitly versioned.

```
{
  "event_id": "uuid",
  "event_type": "incident.created",
  "schema_version": 1,
  "occurred_at": "RFC3339 timestamp",
  "aggregate_type": "incident",
  "aggregate_id": "stable domain ID",
  "correlation_id": "request or workflow ID",
  "payload": { "...": "domain-specific fields" }
}
```

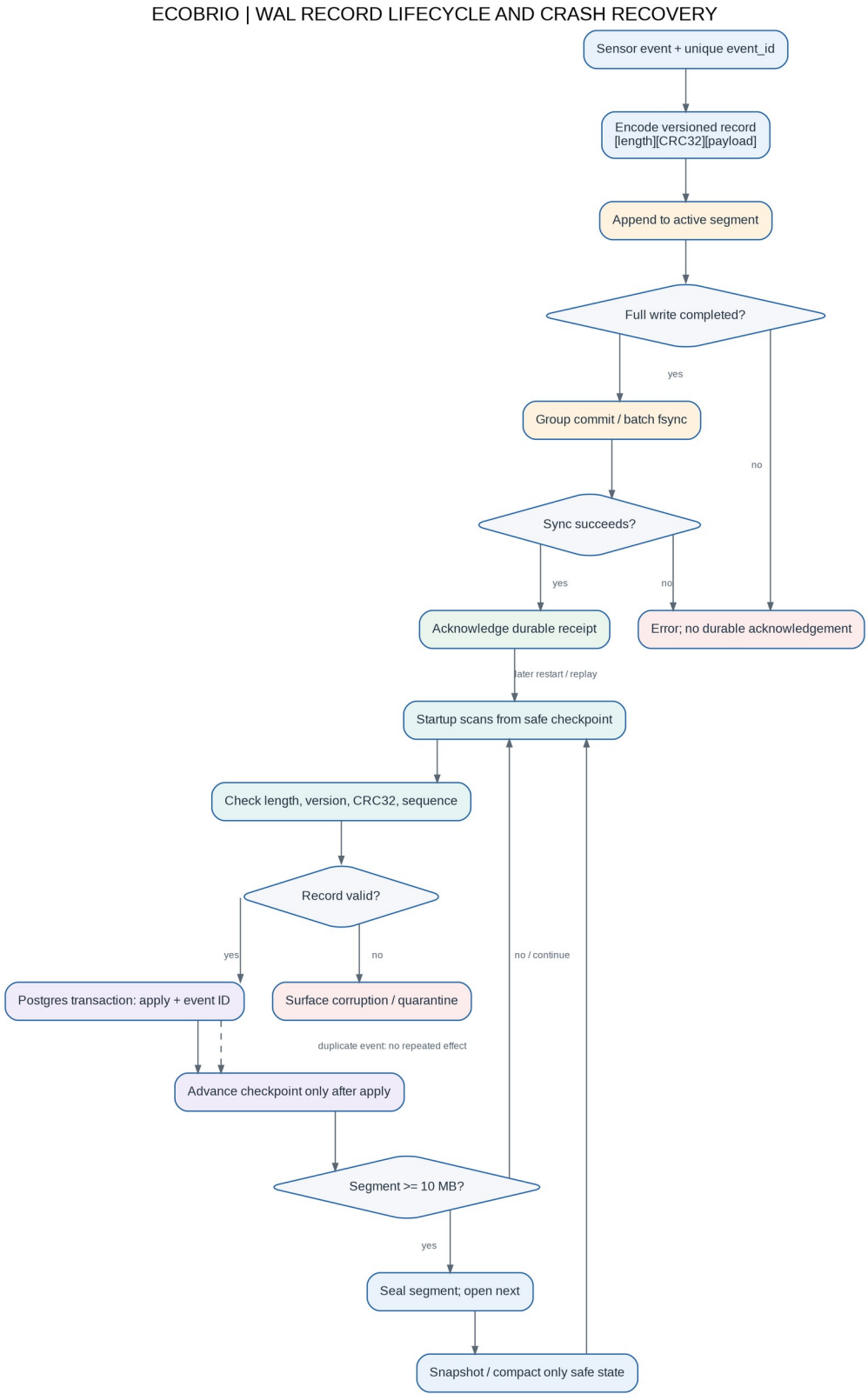
9.2 Outbox protocol

1. Within one PostgreSQL transaction, write the business state and its outbox event.
2. An outbox relay polls pending events and publishes them to LocalStack SQS.
3. After SQS confirms publish, mark the outbox row published. If the relay crashes before marking, it may publish again.
4. Each consumer has its own processed-event identity (inbox/dedup table) and handles business effect plus dedup state transactionally where possible.
5. For side effects outside PostgreSQL, use stable external idempotency identities and a reconciliation process.
6. After bounded attempts, surface failing events in a dead-letter queue or quarantine table and expose them in logs/metrics.

9.3 Suggested event types

Event type	Producer	Consumer / effect
booking.confirmed	Booking service	Notification/analytics worker; must not re-book capacity.
recycling.submission_scanned	Submission service	Inspection workflow / audit trail.
credit.settlement_requested	Credit/ledger service via outbox	Voucher/settlement worker.
credit.settled	Credit service	Citizen history, dashboard projection.
sensor.event_durable	WAL replay/telemetry	PostgreSQL projection and downstream metrics as required.
incident.accepted	Incident intake via outbox	Incident worker to create authoritative incident.
incident.status_changed	Incident service via outbox	Redis GEO projection and live notification relay.

Figure 6. WAL Record Lifecycle and Crash Recovery



The PRD's 10 MB segment rotation and [length:4B][CRC32:4B][payload] record are preserved; event identity and safe checkpointing are recommended additions.

10. WAL Durability and Recovery

10.1 Record contract

Element	Source PRD / recommendation	Reason
Record layout	PRD: 4-byte length, 4-byte CRC32, binary payload. Recommendation: explicit version and event identity in header or payload.	Detect truncation/corruption and support evolution/deduplication.
Byte order	Recommendation: define network/big-endian or another single stable convention.	Same record must parse identically across restarts/builds.
Maximum payload	Recommendation: define a hard max before allocation.	Prevent malformed length from causing unbounded allocation.
Concurrency	Recommendation: one serialized append loop or rigorously synchronized writer.	Prevent interleaved records from concurrent goroutines.
Group commit	PRD: batch fsync flushes. Recommendation: document max batch delay/size and acknowledgement contract.	Balance durability with throughput/latency.
Segment rotation	PRD: 10 MB into immutable historical segments.	Bound active segment size and simplify scanning.
Snapshot/compaction	PRD: compact obsolete sensor states into periodic snapshots. Recommendation: compact only records safely applied/preserved.	Avoid deleting the only recoverable copy of an event.
Recovery	PRD: CRC check, detect torn pages, replay into PostgreSQL. Recommendation: classify tail truncation vs interior corruption.	Avoid silently losing valid records or skipping interior corruption.

10.2 Durability vocabulary

Stage	Meaning	Acknowledge as durable?
Attempted	Application received/accepted work but has not appended a complete record.	No.
Appended	Record bytes were written to the current file, but sync may not have completed.	Only if the contract deliberately provides weaker semantics; not recommended for durable acknowledgement.
Durable	Configured file synchronization succeeded under the documented environment/assumptions.	Yes, under the stated storage contract.
Applied	Logical sensor event committed to PostgreSQL and a deduplication identity is durable.	This is separate from WAL durability; track it independently.

10.3 Recovery algorithm

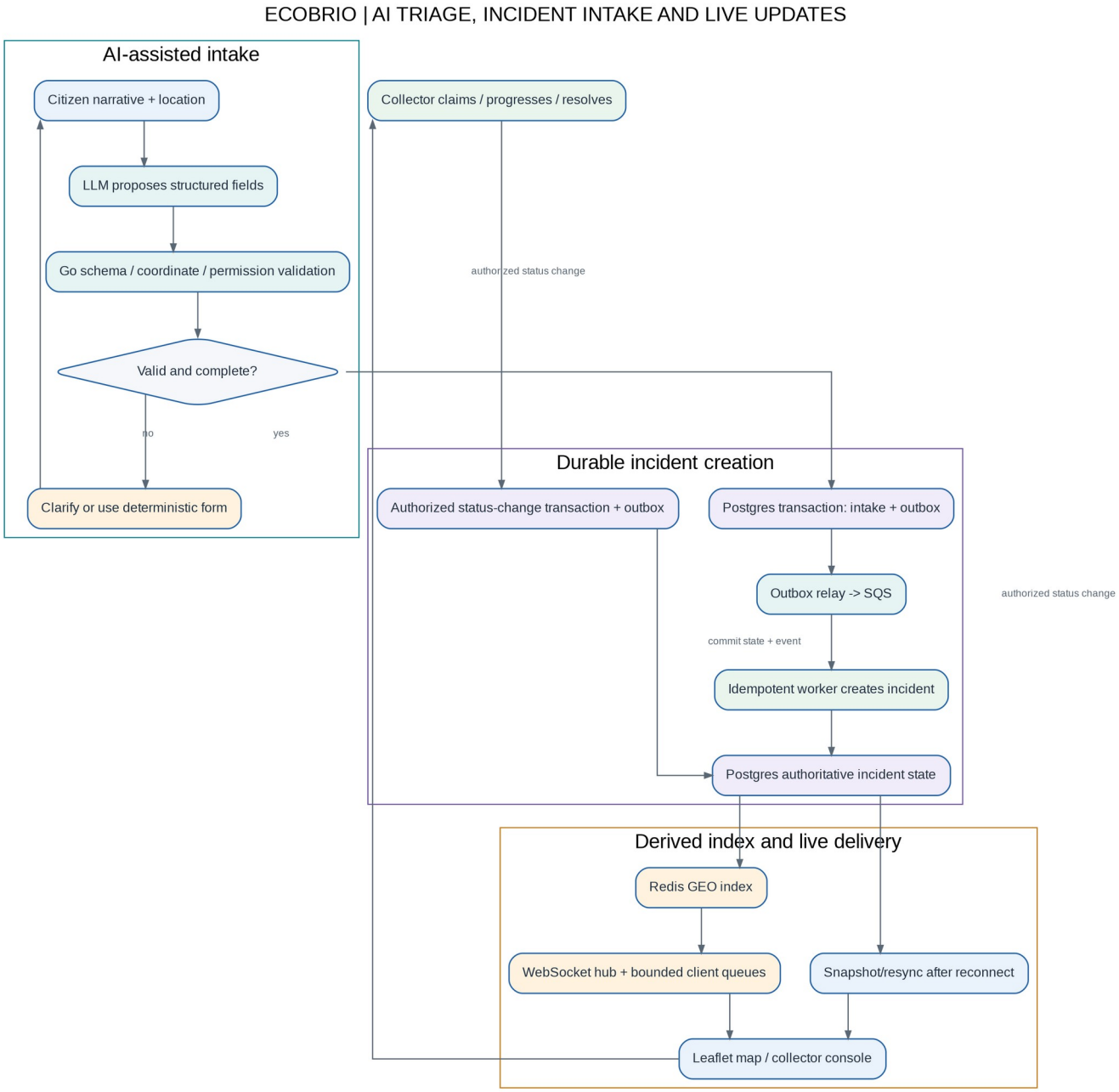
1. At startup, discover and order sealed segments plus the active segment; load the last safe checkpoint/snapshot.
2. Scan records, validate length, version, checksum and sequence/event identity before decoding payload.
3. Treat an incomplete final record according to an explicit torn-tail policy; do not silently skip arbitrary corruption within a sealed segment.

4. Apply each event in a PostgreSQL transaction with a unique event identity. If already applied, skip the logical effect.
5. Advance checkpoint only after the corresponding database application/dedup state is committed.
6. Rotate the active segment at the configured 10 MB threshold; seal it and open the next segment. Snapshots/compaction may remove data only when safe recovery no longer depends on it.
7. Expose recovery duration, scanned records, checksum errors, truncated tails, replayed records, already-applied duplicates, and applied events.

10.4 Crash test matrix

Injection point	Expected outcome
Before append	No durable acknowledgement. No false claim of recoverable event.
During partial record write	Startup detects incomplete tail; preceding valid records remain recoverable.
After append but before required sync	Outcome follows the explicit durability contract; must not be misreported as durably acknowledged.
After successful sync but before PostgreSQL apply	Record is recovered and applied after restart.
After PostgreSQL commit but before WAL checkpoint	Replay detects existing event ID and does not duplicate the logical effect.
Checksum mismatch in active tail	Detect and report according to torn-tail/corruption policy.
Checksum mismatch inside sealed segment	Surface corruption; quarantine/halt safely rather than silently skipping.
During segment rotation/snapshot	Recover a consistent segment sequence and preserve unapplied durable records.

Figure 7. AI Triage, Incident Intake and Live Updates



AI output is validated before use. PostgreSQL is authoritative; Redis GEO is a projection; WebSockets broadcast but do not guarantee delivery to disconnected clients.

11. Geospatial Incidents and WebSockets

11.1 Incident state model

The original PRD describes incident lifecycle states as Raised, In-Progress, and Resolved. A practical implementation may add a Claim/Assigned state if collector assignment is a distinct action. Any additional state should be documented and enforced with a deterministic transition function.

Transition	Guard / side effect
Raised -> In-Progress	Authorized collector claims or starts work; persist actor, time and assignment.
In-Progress -> Resolved	Authorized collector/admin submits resolution details; persist committed result.
Invalid transition	Return typed conflict/validation error; do not mutate state.
Any committed transition	Write outbox event in the same transaction, then update GEO projection and broadcast asynchronously.
Disconnected client reconnects	Fetch authoritative snapshot or resume with sequence/cursor; do not assume Pub/Sub preserved missed events.

11.2 Redis GEO and map

- PostgreSQL stores the authoritative incident coordinates and status. Redis GEO holds the active spatial projection for fast searches.
- Use GEOADD and GEOSEARCH for nearby/viewport queries; define coordinate validation, radius/bounds, result limits and whether resolved items remain indexed.
- Keep map tile usage within the provider's policy; remote OpenStreetMap tiles mean the map is not fully offline even if the backend runs locally.
- Rebuild the GEO index from PostgreSQL after Redis data loss or detected drift.
- Keep incident identifiers and citizen PII out of high-cardinality metrics and broad public broadcasts.

11.3 WebSocket hub rules

- Use bounded per-client outbound queues; a slow client must not block the broadcast loop for every other client.
- Define one writer goroutine per connection or another explicit synchronization policy.
- Handle ping/pong, deadlines, disconnect cleanup, subscription lifecycle, and authentication of subscriptions.
- Add event IDs or monotonically increasing versions so clients can detect gaps.
- Publish committed state changes; provide snapshot/resync behavior for reconnects.
- Benchmark connection capacity and broadcast fan-out separately.

12. KachraSeth AI Triage Agent

KachraSeth converts conversational citizen reports into structured, validated incident actions. The PRD names three deterministic Go tools: report_waste_incident, find_nearest_hub, and get_user_credits, and specifies asynchronous queue dispatch of structured JSON.

12.1 Tool contracts

Tool	Suggested inputs	Permitted behavior	Important guardrail
report_waste_incident	description, reported location, optional coordinates, category suggestion	Propose/submit a report after backend validation	Caller identity and final authorization come from backend context, not LLM output.
find_nearest_hub	validated coordinates or geocoded place	Return nearby eligible hubs/collection points	Do not accept invented or unvalidated coordinates as fact.
get_user_credits	authenticated citizen context	Read the caller's authorized credit summary	Do not accept arbitrary citizen ID without access check.

12.2 AI execution workflow

1. Receive citizen text and required context from the authenticated session.
2. Ask the LLM to produce structured fields or select a permitted tool; use typed JSON/schema validation.
3. Reject malformed, incomplete, unauthorized, or out-of-scope tool calls. Ask for clarification if location or category is ambiguous.
4. Execute tools in Go under the caller's permissions; never let the model directly access the database or bypass domain services.
5. For incident creation, persist accepted intake and outbox event before queue processing; distinguish "received", "processing", and "incident created" in the UI.
6. Apply timeout, rate-limit, retry, and provider-unavailable behavior. Keep a deterministic manual incident form so core reporting works without AI.
7. Do not log API keys or unnecessary sensitive citizen text. Record model/provider and outcome metadata safely for troubleshooting.

> **Zero-cost qualification**
>
> LocalStack and Docker can keep infrastructure local. A Gemini/Groq API may still require internet access and may be subject to provider quotas or pricing changes; public map tiles also depend on remote tile infrastructure. Keep both optional and provide deterministic/manual fallbacks.

13. Data Model, API, and Event Contracts

13.1 Recommended table inventory

Table	Core fields / responsibilities	Key constraints
users	user_id, role, profile/status	Role checks and stable identity.
collection_hubs	hub_id, name, latitude, longitude, metadata	Valid coordinates; optionally unique external/hub code.
collection_slots	slot_id, hub_id, total_capacity, confirmed_count or derived count, version	Capacity >= 0; database-enforced reservation invariant.
bookings	booking_id, citizen_id, slot_id, state, request_key, timestamps	Unique request identity in scope; FK to slot/user.
recycling_submissions	submission_id, QR identity, citizen_id, material type/weight, inspection state, inspector	QR/submission uniqueness; state transition checks.
credit_accounts	account_id, citizen_id, optional cached balance	Balance is projection; ledger is authoritative.
ledger_entries	entry_id, transaction_id, account_id, side, integer points, timestamp, reversal_of	Unique business transaction/effect; immutable postings.

Table	Core fields / responsibilities	Key constraints
idempotency_records	scope, key, request fingerprint, status, response/business reference	Unique scoped key; conflicting payload reuse rejected.
vouchers / redemptions	business transaction ID, provider ID, status, request/response metadata	Unique idempotency identity to provider; reconciliation status.
incidents	incident_id, description/category, coordinates, severity, status, assignee, timestamps	Valid lifecycle and authorized state transitions.
sensor_events	event_id, sensor_id, sequence, payload/reference, applied status	Unique event ID or sensor-sequence key.
outbox_events	event_id, event_type, schema_version, payload, created/published timestamps, attempts	Unique event ID; pending rows retained until published.
processed_events	consumer_name, event_id, processed_at	Unique consumer/event pair.

13.2 Suggested API map

Area	Example endpoints (proposed)	Notes
Health	GET /healthz; GET /readyz	Liveness vs. dependency readiness.
Hubs	GET /api/v1/hubs?lat=&lon=&radius=	Coordinates validated; return bounded results.
Slots	GET /api/v1/hubs/{hubId}/slots	Availability is an estimate until booking token/transaction completes.
Bookings	POST /api/v1/bookings; GET /api/v1/bookings/{id}; GET /api/v1/bookings/by-request/{key}	Require idempotency on creation; stable outcome response.
Submissions	POST /api/v1/submissions; POST /api/v1/submissions/{id}/inspection	Inspection transition authorized to collector/admin.
Credits	GET /api/v1/credits/me; GET /api/v1/credits/transactions	Return persisted account/ledger history; protect identity access.
Incidents	POST /api/v1/incidents; GET /api/v1/incidents; PATCH /api/v1/incidents/{id}/status	Creation and transitions validate roles and state machine.
AI	POST /api/v1/triage; POST /api/v1/ai/tools/{toolName} (internal only)	Keep tool endpoints internal/server-side; strict allowlist.
Telemetry	POST /api/v1/telemetry/events	Response must communicate received vs. durable status correctly.
Metrics	GET /metrics	Expose Prometheus metrics on controlled internal interface.

API status semantics

Use a clear status vocabulary. “Accepted for processing” is not the same as “committed”, and “committed” is not the same as “broadcast to every WebSocket client”. Return stable request IDs and make the next status inspectable.

13.3 Event schemas and correlation

Every event should include event_id, event_type, schema_version, occurred_at, aggregate_type, aggregate_id, correlation_id and payload. Include producer/version metadata if it helps debugging. Avoid putting secrets or unnecessary personal data into queue messages and telemetry logs.

14. Security, Privacy, and Reliability

Control	Implementation requirement	Verification
Authentication	Secure sessions or JWT with explicit token lifetime and validation.	Unauthenticated request tests; expired/invalid token tests.
Authorization / RBAC	Citizen, collector and municipal admin permissions checked per action.	Role matrix integration tests; attempt cross-user access.
QR replay prevention	Unique submission identity; server-side state and uniqueness constraints.	Replay identical QR submission concurrently; one logical submission/reward.
AI tool permissions	Tool allowlist; identity sourced from authenticated request; schema checks.	Prompt injection / invalid tool payload / privilege escalation tests.
Secrets	Environment variables locally; secret values excluded from source, client JS and logs.	Secret scanning; .env.example contains placeholders only.
API abuse	Request-size limits, rate limits, deadlines, bounded retries.	Oversized payload and abuse-rate tests.
PII and location privacy	Minimize precise location retention and public data; redact logs where appropriate.	Review sample logs and websocket payloads.
Database integrity	Foreign keys, unique constraints, transactions, least-privilege DB user.	Negative tests for invalid state and duplicate effects.
Queue reliability	Idempotent consumers, visibility timeout, DLQ/quarantine, bounded retries.	Duplicate message and repeated failure tests.
WAL file safety	Persistent directory, validated record bounds, corruption policy, safe permissions.	Truncation/checksum/corrupt-header tests.
Dependency outages	AI provider, Redis, queue and map failures have explicit behavior.	Fault injection and deterministic fallback checks.

15. Observability and Performance Benchmarks

15.1 Metric inventory

Metric name (suggested)	Type / dimensions	Purpose
ecobrio_request_duration_seconds	Histogram; bounded route and status labels	API latency distribution; p50/p95/p99 from buckets.
ecobrio_requests_total	Counter; route template/status class	Rate, request mix and error ratio.
ecobrio_booking_conflicts_total	Counter; reason class	Contention visibility.
ecobrio_booking_compensation_total	Counter; outcome class	Redis/SQL failure and release monitoring.
ecobrio_idempotency_hits_total	Counter; operation type	Retry behavior / duplicate request rate.
ecobrio_wal_records_appended_total	Counter	Input records written to WAL.
ecobrio_wal_records_durable_total	Counter	Records crossing the defined durability boundary.
ecobrio_wal_records_replayed_total	Counter; replay outcome	Recovery workload.
ecobrio_wal_corruption_total	Counter; corruption class	Integrity failures surfaced.
ecobrio_websocket_connections	Gauge	Current connection count.
ecobrio_websocket_broadcast_duration_seconds	Histogram; fan-out bucket or event type	Publisher-to-receiver latency as defined in benchmark.
ecobrio_queue_messages_pending	Gauge; queue name	Backlog and worker lag.
ecobrio_outbox_oldest_pending_seconds	Gauge/histogram	Outbox relay delay.

Metric name (suggested)	Type / dimensions	Purpose
ecobrio_geo_query_duration_seconds	Histogram; query type	Spatial index latency by workload.

Metric cardinality rule

Use route templates and bounded outcome classes. Do not use unique user IDs, booking IDs, incident IDs or request IDs as Prometheus labels; put high-cardinality details in structured logs or traces instead.

15.2 Benchmark protocol

1. Record hardware/OS, Go version, container limits, PostgreSQL/Redis/LocalStack versions, and whether load generator and application share a host.
2. Define warm-up, measurement duration, target arrival rate, concurrency, payload size, success/error mix, and dropped-iteration accounting.
3. Run the booking correctness race test independently of the throughput test.
4. Report total requests, successful reservations, conflicts, internal errors, request rate, p95/p99, and dropped iterations.
5. Benchmark GEO with stated index size, query geometry/radius, returned result count and percentile.
6. Benchmark WebSocket connection scale separately from fan-out; state if all clients or only geographic subscribers receive every event.
7. For WAL, compare the complete expected event set with appended, durable, replayed, duplicate-skipped, and applied event IDs after each fault scenario.
8. Save the exact k6 scripts, Go test output, Prometheus query/results, Grafana screenshots and a concise environment report in the repository.

15.3 Interpreting the PRD numbers

The targets are ambitious and workload-dependent. A booking endpoint can appear fast if it rejects most traffic because slots are full; report successful bookings and conflicts separately. A 10,000-connection idle WebSocket test is not equivalent to 10,000 subscribers receiving high-rate fan-out. A process kill does not simulate every hardware or power-loss failure. These limitations should be stated in the benchmark report.

16. Test Strategy and Acceptance Matrix

16.1 Test layers

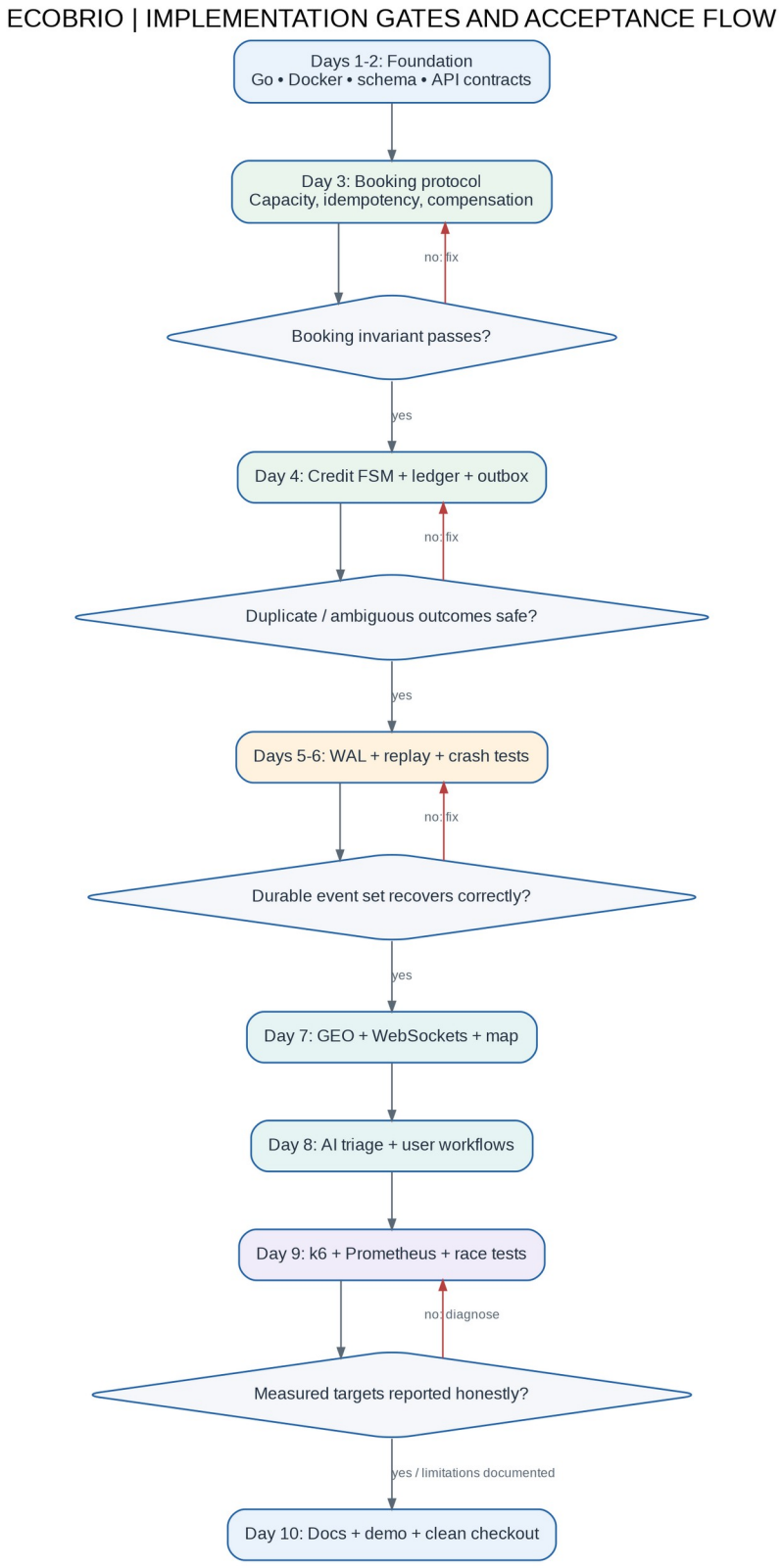
Test layer	What to test	Primary tooling
Unit	FSM transitions, checksum/encoding, capacity helpers, request fingerprinting, retry decisions.	Go testing; table-driven tests.
Integration	Go service + real local PostgreSQL/Redis/LocalStack.	Go testing, test containers or Compose test profile.
Concurrency	Many callers, shared slots, duplicate keys, channel close/shutdown paths.	Go tests under -race; k6 for API contention.
Fault injection	SQL timeout, Redis loss, queue duplicate, process kill, partial WAL record.	Controlled chaos/integration scripts.
Load	Throughput, latency, conflict mix, queue lag, WebSocket scale.	k6, Prometheus, browser/client listener.

Test layer	What to test	Primary tooling
End-to-end	Citizen books, submits waste, earns/redeems points, reports incident and sees map update.	Scripted browser/API scenario.
Recovery regression	Reopen WAL, replay duplicate events, rebuild GEO, reconcile reservation tokens.	Deterministic fixtures and recovery scripts.

16.2 Acceptance-test matrix

ID	Scenario	Pass condition
B1	2,000 concurrent booking attempts for five places	No more than five confirmed bookings.
B2	Same booking idempotency key repeated concurrently	One logical booking and stable original outcome.
B3	Redis claim succeeds and SQL rejects	Reservation release/reconciliation happens at most once.
B4	SQL commits but client sees timeout	Retry resolves to existing booking.
B5	Redis counter is lost/drifts	Rebuild/reconciliation restores derived state without changing confirmed booking truth.
C1	Same credit request repeated/concurrent	One logical ledger posting.
C2	Invalid FSM transition	Typed error and no unintended ledger effect.
C3	Voucher success response is lost	No duplicate voucher; outcome reconciled by stable external business ID.
C4	Voucher failure confirmed	Escrow/hold compensation applied once.
E1	Relay crashes after publishing but before marking sent	Duplicate publication is safe; consumer deduplicates.
E2	Worker receives same SQS message twice	One logical side effect.
E3	Repeated worker failure	Message becomes inspectable via bounded retry / DLQ or quarantine.
W1	Truncated final WAL record	Valid earlier records recover; tail handled explicitly.
W2	CRC mismatch inside sealed segment	Corruption surfaced; no silent skip.
W3	PostgreSQL commit before checkpoint crash	Replay does not apply duplicate logical effect.
W4	Crash before/after sync boundary	Acknowledged durable set matches recovery contract.
G1	Incident updates while a client is disconnected	Reconnect snapshot/cursor recovers current state.
G2	Slow WebSocket client	Other subscribers continue receiving updates.
A1	Invalid model output or provider outage	No unauthorized write; user has safe fallback.
P1	Sustained rate-controlled booking load	Rate, latency, errors, success/conflict mix and dropped iterations recorded.
P2	go test -race ./...	No races detected in exercised paths; output retained.

Figure 8. Implementation Gates and Acceptance Flow



Do not proceed to high-scale benchmarking until booking, ledger and WAL invariants pass. If a gate fails, fix and rerun the relevant tests before continuing.

17. Ten-Day Implementation Roadmap

Day	Milestone	Tasks	Exit gate
Day 1	Architecture and local setup	Go module; Docker Compose for PostgreSQL, Redis, LocalStack; migrations; config; directory layout; baseline contracts.	docker compose up succeeds; migrations and health checks pass.
Day 2	API and data foundation	HTTP middleware; logs; request IDs; hubs/slots/incidents basic CRUD; DB constraints; integration test scaffolding.	CRUD/integration tests run against local services.
Day 3	Booking correctness	Redis Lua reservation token; SQL conditional capacity/version update; idempotency; safe compensation; reconciliation.	Capacity invariant and uncertain-commit tests pass.
Day 4	Credit FSM and event delivery	FSM; PostgreSQL idempotency; double-entry ledger; chosen voucher semantics; outbox; SQS worker.	Duplicate requests and failure paths preserve ledger consistency.
Day 5	WAL record engine	Versioned record contract; append; CRC32; batch flush; sync boundary; torn-record tests.	Valid record append/reopen works; malformed records are detected.
Day 6	WAL recovery	Segment rotation at 10 MB; snapshots; checkpoints; replay; dedupe; crash injection.	Durable event set recovers correctly in defined test scenarios.
Day 7	Geo + real-time UI	Redis GEO; incident status transitions; WebSocket hub; map; reconnect/snapshot behavior.	Multiple tabs receive committed updates; reconnect recovers current state.
Day 8	AI + citizen workflow	KachraSeth structured tools; validation; fallbacks; SQS intake; booking/credits/reporting screens.	Citizen report becomes a validated incident through the asynchronous flow.
Day 9	Observability and benchmarks	Prometheus/Grafana; k6 load/race workloads; -race; WAL chaos tests; latency and recovery report.	Every PRD target has a reproducible result or documented limitation.
Day 10	Hardening and portfolio release	Clean-checkout run; fix critical defects; README; diagrams; OpenAPI; benchmark report; demo recording.	Reviewer can start the system and reproduce core verification tests.

17.1 If schedule slips

Reduce UI complexity and defer AI polish before weakening correctness. Preserve booking invariant, ledger idempotency, WAL replay correctness, and integration tests. Treat 3,000 requests/sec, 10,000 WebSocket connections and sub-15 ms broadcast as scale targets to measure; do not claim success simply because the code runs.

18. Risk Register and Architecture Decisions

ID	Risk	Severity	Failure mode	Mitigation
R1	Redis/PostgreSQL drift	High	Reservation token and committed booking disagree after timeout/failure.	Unique tokens, SQL capacity invariant, idempotent release, periodic reconciliation.

ID	Risk	Severity	Failure mode	Mitigation
R2	Duplicate credit/voucher effect	High	Retries, queue redelivery or ambiguous provider response.	Persisted idempotency, stable external transaction IDs, double-entry ledger, reconciliation.
R3	WAL data loss or duplicate replay	High	Undefined ack boundary or commit-before-checkpoint crash.	Durability contract, event IDs, idempotent apply, fault-injection matrix.
R4	Outbox dual-write gap	High	Business data commits but queue event never publishes.	Transactional outbox relay, pending-event metric, idempotent consumers.
R5	WebSocket message loss / slow consumer	Medium-high	Disconnect or blocked broadcast loop.	Per-client bounded queues, snapshots/resync, fan-out benchmark.
R6	AI malformed / unsafe tool call	Medium-high	Incorrect category/location or privilege attempt.	Strict schema, tool allowlist, user authorization, manual fallback.
R7	Unverifiable performance claim	High	Targets quoted as achieved without measurements.	Versioned scripts, raw outputs, environment details, target/result distinction.
R8	Scope overload in ten days	High	Complex engines and UI compete for time.	Vertical slices, gates, simplify UI/AI first.
R9	False zero-cost/offline claim	Medium	External AI endpoint or remote map tiles unavailable.	Optional external integrations; deterministic fallback; document dependency.
R10	Metrics cardinality explosion	Medium	User/event IDs used as labels.	Bounded labels; IDs only in logs/traces.

18.1 Architecture decisions to lock

Decision	Recommended baseline	Must be decided / documented
Booking truth	PostgreSQL authoritative; Redis token for fast reservation admission.	Token TTL, reservation states, SQL capacity mechanism, timeout resolution and reconciliation frequency.
Credit-voucher meaning	Option A for strict PRD fidelity; Option B for separate points wallet/redemption.	Choose one and align FSM, ledger semantics, UI labels and compensation.
WAL durability	Durable acknowledgement only after configured sync succeeds.	Batch delay/size, record version, max payload, torn-tail and interior corruption behavior.
Event semantics	Outbox + at-least-once delivery + idempotent consumers.	Event versioning, consumer identity, retry count, DLQ/quarantine ownership.
Incident event consistency	Postgres canonical; GEO index and WebSocket delivery derived.	Outbox use, event sequence/cursor, snapshot API, location subscription rules.
AI availability	Provider interface plus structured form fallback.	Timeout, quota/error UI, whether AI is required for any core business action.
Performance claim policy	Target vs measured result separated in docs and slides.	Machine/environment details and benchmark artifacts required before claim.

19. Repository Structure and Local Developer Workflow

```
ecobrio/
├── cmd/
│   └── server/main.go
├── internal/
│   ├── booking/
│   ├── credits/
│   ├── ledger/
│   ├── telemetry/
│   │   ├── wal/
│   │   ├── recovery/
│   │   └── snapshots/
│   ├── incidents/
│   ├── geospatial/
│   ├── websocket/
│   ├── ai/
│   ├── queue/
│   ├── outbox/
│   ├── database/
│   ├── middleware/
│   └── observability/
├── migrations/
├── scripts/
│   ├── load-tests/
│   └── chaos-tests/
├── tests/
│   ├── integration/
│   └── end-to-end/
├── web/
│   ├── citizen/
│   ├── collector/
│   └── admin/
├── deploy/
│   ├── docker-compose.yml
│   ├── localstack/
│   ├── prometheus/
│   └── grafana/
├── docs/
│   ├── architecture/
│   ├── api.md
│   ├── data-model.md
│   └── benchmarks.md
├── .env.example
├── Makefile
├── go.mod
└── README.md
```

19.1 Suggested developer commands

```
# Local services and application
cp .env.example .env       # fill safe local values; never commit secrets
docker compose up --build
go test ./...
go test -race ./...
go vet ./...

# Suggested Makefile aliases (implement them in the repo)
make up
make migrate
make test
make test-race
make test-integration
make bench-booking
make chaos-wal
make down
```

The exact Makefile commands are proposed conventions, not existing commands unless implemented. A clean-checkout test must verify the documented instructions before release.

20. Demonstration Plan, Interview Positioning, and Delivery Checklist

20.1 Five-minute demonstration

Time	Demo action	Evidence to show
0:00-0:40	Show the EcoBrio citizen/operator UI and identify the four main engines.	Architecture diagram; hub/slot list; incident map.
0:40-1:30	Run a booking race against five remaining slots.	k6 summary, confirmed booking count, conflicts, zero over-allocation assertion.
1:30-2:20	Replay a duplicate QR/credit request or simulate voucher failure.	Idempotency result, ledger transaction identity, audit trail and compensation/reconciliation state.
2:20-3:20	Ingest sensor records, stop the process at a controlled point, restart and replay.	Expected vs durable vs replayed vs applied event IDs; checksum/recovery output.
3:20-4:10	Submit a natural-language waste report and watch it become a validated incident.	Structured tool output, outbox/SQS event, database row, GEO/map update.
4:10-5:00	Show Grafana and benchmark report.	Latency percentiles, request mix, environment, raw result artifacts and limitations.

20.2 Interview explanation template

> **Use only after replacing placeholders with measured results**
>
> EcoBrio is a Go-based reverse-logistics and geospatial telemetry platform. I focused on correctness under contention and failure: the booking engine combines Redis atomic reservation tokens with PostgreSQL capacity enforcement and reconciliation; the incentive workflow uses persisted idempotency and an auditable double-entry ledger; and sensor ingestion uses a custom segmented WAL with CRC32

> verification and replay. I verified these guarantees with [specific test suite] and measured [actual throughput/p99/recovery results] on [documented environment]. The platform also provides geospatial incident tracking, WebSocket updates, and validated AI-assisted triage.

Do not fill in the bracketed values with PRD targets unless the corresponding tests actually pass and the raw output is available. Mention the exact test limitations when describing WAL crash recovery or WebSocket scale.

20.3 Final delivery checklist

- ☐ Go backend builds from a clean checkout.
- ☐ Docker Compose starts PostgreSQL, Redis and LocalStack reliably.
- ☐ Database migrations and seed/demo data work repeatably.
- ☐ Booking capacity invariant passes the 2,000-attempt / five-slot test.
- ☐ Booking idempotency, uncertain commit, compensation and reconciliation tests pass.
- ☐ Credit FSM rejects invalid transitions and duplicate requests do not create duplicate ledger effects.
- ☐ Ledger entries are balanced, auditable and corrected through explicit reversals.
- ☐ WAL format, CRC32, sync, rotation, replay and checkpoint behavior are documented.
- ☐ WAL crash/fault matrix is repeatable; durable event set is reconciled after recovery.
- ☐ Outbox and SQS workers tolerate duplicate delivery and repeated failure.
- ☐ Incident map displays committed state; reconnect resync works.
- ☐ AI tool calling is validated and manual incident reporting works with AI disabled.
- ☐ Prometheus/Grafana dashboards expose the key correctness and performance signals.
- ☐ k6 scripts include workload config, request mix and dropped-iteration checks.
- ☐ Go race tests, unit tests and integration tests are available in CI.
- ☐ README separates PRD targets, measured results, known limitations and non-goals.
- ☐ Architecture and workflow diagrams match the implementation.
- ☐ Demo script does not depend on manual database edits or hidden setup steps.

21. References and Glossary

21.1 Source document

EcoBrio: System Architecture & PRD — Distributed Reverse-Logistics, Geospatial Telemetry & Incentive Clearing Engine (4 pages). This source is reproduced without modification in Appendix A. Source page mapping: overview and stack (page 1); engineering engines (page 2); targets and roadmap (page 3); interview pitch (page 4).

21.2 Technical references for implementation

Redis Lua scripting: https://redis.io/docs/latest/develop/programmability/eval-intro/ — Atomic script execution within Redis; not cross-store atomicity with PostgreSQL.

PostgreSQL WAL introduction: https://www.postgresql.org/docs/16/wal-intro.html — PostgreSQL's own WAL and its role in database durability/recovery.

Amazon SQS Standard queues: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues.html — At-least-once delivery and need for idempotent consumers.

Prometheus histograms: https://prometheus.io/docs/practices/histograms/ — Histogram buckets and latency quantile calculation.

Go race detector: https://go.dev/doc/articles/race_detector — Race detector scope and execution-path limitations.

k6 constant-arrival-rate executor: https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/ — Rate-controlled load generation independent of response time.

OpenStreetMap tile usage policy: https://operations.osmfoundation.org/policies/tiles/ — Public tile service usage and attribution/caching expectations.

21.3 Glossary

Term	Meaning in EcoBrio
ACID	Atomicity, consistency, isolation and durability of database transactions.
Compensation / saga	A forward-recovery action that restores the business invariant when a multi-step workflow cannot complete normally.
CRC32	A checksum used to detect many record corruption cases; not a cryptographic integrity guarantee.
FSM	Finite state machine; a set of named states and explicitly permitted transitions.
Idempotency	Repeated execution of the same logical request does not create additional logical side effects.
Inbox / processed-event table	A durable consumer-side record used to deduplicate repeated event delivery.
Outbox	A database table written in the same transaction as business state, then relayed asynchronously to a message queue.
Optimistic locking	A conditional update using a version/expected prior state to detect concurrent modification.
p99 latency	A latency percentile below which 99% of measured requests fall for the defined window/workload.
Redis GEO	Redis geospatial index commands used for proximity/radius/viewport-oriented lookup.
WAL	Write-ahead log; a durable append-oriented record of changes/events that can be replayed after restart.

Appendix A. Original EcoBrio PRD (Reproduced)

The original four-page PDF supplied for this project follows this divider unchanged. It is included to preserve the source requirements, architecture rationale, success targets, ten-day roadmap and original interview pitch in one consolidated artifact. The analytical recommendations in the preceding sections do not modify the original source pages.

> **Source baseline preserved**
>
> For any disagreement, consult the original PRD pages in the appended source copy. Requirements introduced as “recommended” in this blueprint are design refinements and should be treated as proposals, not as wording present in the source PRD.

EcoBrio: System Architecture & PRD

Distributed Reverse-Logistics, Geospatial Telemetry & Incentive Clearing Engine

Target Audience: Tier-1 Product Companies (Rubrik, JusPay, Myntra, Booking.com)	**Budget Constraints:** ■0 / $0 (100% Free / LocalStack)
Core Language: Go (Golang 1.22+)	**Architecture Pattern:** Distributed Event-Driven & FSM

1. Executive Summary & Problem Scope

Urban circular economies struggle with fragmented reverse-logistics: irregular informal recycler routing, hazardous e-waste contamination, and inaccurate incentive allocation. **EcoBrio** solves this by providing a high-throughput, crash-resilient distributed backbone that orchestrates municipal collection hubs, tracks smart-bin telemetry, settles tokenized green incentives, and renders real-time incident resolution maps.

Strategic Objective: Designed to serve as an irrefutable engineering portfolio piece demonstrating low-level concurrency, crash recovery via Write-Ahead Logging (WAL), atomic distributed locking, and event-driven cloud streaming.

2. Zero-Cost Technology Stack

Component	Technology Choice	Engineering Rationale & Free-Tier Footprint
Primary Backend	Go (Golang 1.22+)	Lightweight goroutines (2KB overhead), native channels, zero GC bloat. Compiles to native binary.
Relational DB	PostgreSQL 16	ACID double-entry ledger & row versioning (@Version) for optimistic locking. Dockerized local instance (■0).
In-Memory & Geo	Redis 7 (Alpine)	Atomic Lua scripts for capacity locking, Redis GEO for <2ms spatial radius queries, and Pub/Sub. Docker (■0).
Cloud Mock / Emulation	LocalStack (S3, SQS)	Simulates AWS S3 presigned URLs & SQS event buffering completely offline on localhost with zero cloud billing (■0).
Storage Durability	Segmented WAL (Custom Go)	Append-only binary log with fsync() batches and CRC32 verification for crash resilience.
Observability	Prometheus + Grafana	Tracks golden signals: p99 latency, request throughput, active WebSockets, lock collisions (Docker).
AI Triage Agent	Gemini Free / Groq API	Deterministic function/tool calling for unstructured citizen incident reporting with zero API fees.

3. Detailed Engineering Engines

3.1 Concurrency-Safe Kiosk Slot Booking Engine (Myntra / Booking.com Focus)

Challenge: Prevent overselling when hundreds of residents simultaneously attempt to book limited e-waste drop-off slots at neighborhood kiosks.

- **Layer 1 (Redis Lua Atomic Decrement):** Checks and decrements available slot capacity in a single atomic cycle to deflect 95% of invalid requests before hitting SQL.
- **Layer 2 (Postgres Optimistic Locking):** Confirms reservation with versioned SQL updates (WHERE id = $1 AND version = $2). On collision, attempts up to 3 retries or returns HTTP 409 Conflict.
- **Validation Bar:** 0% overselling across 2,000 concurrent simulated booking attempts in k6.

3.2 Idempotent Credit Clearinghouse & State Machine (JusPay Focus)

Challenge: Citizens surrendering recyclables scan QR passports over intermittent mobile 4G. Retried network requests must never mint duplicate credits.

- **Idempotency Gate:** Client sends X-Idempotency-Key: <UUID>. Redis intercepts retries and returns cached responses without re-executing credit transactions.
- **Finite State Machine (FSM):** Enforces strict transitions: SCANNED → INSPECTION_PASSED → ESCROW_LOCKED → CREDITED. Invalid transition attempts fail with typed errors.
- **Saga Compensating Rollbacks:** If downstream voucher issuance (transit/EV pass) fails, automated rollbacks void locked escrow credits and restore account integrity.

3.3 Crash-Resilient Sensor Ingestion with Segmented WAL (Rubrik Focus)

Challenge: Thousands of smart compactors and solar beacons emit high-frequency telemetry. Uncommitted writes must survive sudden server crashes (SIGKILL).

- **Segmented Append-Only Log:** Every sensor record is formatted as [Length: 4B][CRC32: 4B][Binary Payload] and appended to disk with batch fsync() flushes.
- **Compaction & Rotation:** WAL logs rotate at 10MB into immutable historical segments; background workers compact obsolete sensor states into periodic snapshots.
- **Crash Replay Daemon:** On server startup, verifies CRC32 checksums, detects torn pages, and replays uncommitted sensor updates directly into PostgreSQL.

3.4 Real-Time Geospatial Incident Tracker & WebSocket Hub

Challenge: Track city waste incidents across life stages (Raised, In-Progress, Resolved) with real-time browser map visualization without hammering the relational DB.

- **Redis GEO Index:** Ingests incident coordinates via GEOADD. Viewport boundary queries return active issues in sub-2ms time via GEOSEARCH.
- **High-Concurrency WebSocket Hub:** Written in native Go using channels and goroutines; supports 10,000+ persistent connections with under 15ms broadcast latency.
- **Zero-Cost Map UI:** Built with Leaflet.js and OpenStreetMap tiles; color-coded pins update live as collectors claim or resolve incidents.

3.5 KachraSeth: Autonomous AI Triage Agent

Challenge: Convert conversational citizen reports into structured, validated incident payloads without manual municipal triage.

- **Deterministic Tool Calling:** Binds Gemini / Groq LLMs to Go functions: report_waste_incident, find_nearest_hub, and get_user_credits.

- **Asynchronous Queue Dispatch:** Structured JSON outputs are published directly to Amazon SQS / LocalStack for worker consumption, ensuring the chat interface never blocks on backend writes.

4. Performance Targets & Success Verification

Metric / Feature	Target Threshold	Verification Mechanism
Peak Booking Throughput	≥ 3,000 Requests/Sec	k6 load test script (500 virtual users against localhost)
p99 Latency	< 40ms on reservation APIs	Prometheus histogram metric (ecobrio_request_duration_seconds)
Concurrency Safety	0% Over-allocation (0 race conditions)	Concurrent k6 booking race test against last 5 available slots
Durability & Recovery	100% byte integrity (Zero loss)	Chaos test: kill -9 during 5k sensor writes + restart recovery test
Live Broadcast Latency	< 15ms end-to-end	Benchmarked WebSocket client telemetry listener
Code Concurrency Safety	Zero data races detected	Enforced via go test -race ./... in CI/CD pipeline

5. 10-Day Implementation Roadmap

Days	Milestone Deliverables	Verification Gate
Days 1–2	Go scaffolding, PostgreSQL migrations, Docker Compose (Postgres, Redis, LocalStack)	docker compose up runs cleanly with zero errors
Days 3–4	Redis Lua scripts for atomic slot booking, X-Idempotency-Key middleware, Ledger FSM	Unit tests verify duplicate request prevention and slot integrity
Days 5–6	Custom Segmented WAL engine with CRC32 checksums, fsync() batching, and recovery daemon	Passing chaos_test.go simulating sudden process termination
Day 7	Redis GEO spatial indexing, Go WebSocket broadcast hub, Leaflet.js live interactive map UI	Map pins update in real-time across multiple open browser tabs
Day 8	KachraSeth LLM integration via tool/function calling; SQS event queue buffering	Chatbot outputs structured JSON ticket payloads into LocalStack SQS
Day 9	k6 concurrency benchmarks, Prometheus & Grafana dashboard setup, latency metrics capture	Grafana dashboard displays real-time 3,000+ RPS benchmark metrics
Day 10	GitHub repository finalization: architecture diagram, benchmark charts, production README	Clean, zero-click runnable repo ready for resume linking

6. Professional Interview Positioning (The 60-Second Pitch)

> *"EcoBrio is a distributed reverse-logistics and geospatial telemetry engine built in Go. The primary systems challenge was guaranteeing zero slot overselling during high-contention e-waste drop-off bursts while maintaining transactional ledger consistency and crash durability. I resolved this using atomic Redis Lua scripts coupled with PostgreSQL optimistic locking, an append-only segmented Write-Ahead Log with CRC32 verification, and a high-concurrency Go WebSocket hub delivering real-time geospatial incident streaming at under 15ms latency. All benchmarks were validated under automated 3,000+ RPS load testing using k6."*